Below is a **complete ASP.NET Core Web API project** that includes:

✓ Custom Middleware
✓ Built-in Middleware
✓ Controller
✓ Dependency Injection
✓ Program.cs pipeline
✓ Explanation of every part

This is a **ready-to-run example** for beginners and interviews.

✅ Project Name: MiddlewareDemo

Folder Structure:

```
MiddlewareDemo/
 ├── Middlewares/
 |    └── RequestLoggingMiddleware.cs
 |    └── BlockSecretMiddleware.cs
 ├── Controllers/
 |    └── HomeController.cs
 ├── Program.cs
 └── appsettings.json
```

🧱 Step 1 — Create Custom Middleware #1

👉 RequestLoggingMiddleware

Logs incoming requests + outgoing responses.

Middlewares/RequestLoggingMiddleware.cs

```
using Microsoft.AspNetCore.Http;


namespace MiddlewareDemo.Middlewares
```

```
{
    public class RequestLoggingMiddleware
    {
        private readonly RequestDelegate _next;

        public RequestLoggingMiddleware(RequestDelegate next)
        {
            _next = next;
        }

        public async Task InvokeAsync(HttpContext context)
        {
            Console.WriteLine($"➡ Request: {context.Request.Method} {context.Request.Path}");

            await _next(context);

            Console.WriteLine($"⬅ Response: {context.Response.StatusCode}");
        }
    }

    // Extension method for clean code
    public static class RequestLoggingMiddlewareExtensions
    {
        public static IApplicationBuilder UseRequestLogging(this IApplicationBuilder app)
        {
            return app.UseMiddleware<RequestLoggingMiddleware>();
```

```
        }
    }
}
```

🧱 **Step 2 — Create Custom Middleware #2**

👉 **BlockSecretMiddleware**

Blocks requests to /secret route.

Middlewares/BlockSecretMiddleware.cs

```
using Microsoft.AspNetCore.Http;

namespace MiddlewareDemo.Middlewares
{
    public class BlockSecretMiddleware
    {
        private readonly RequestDelegate _next;

        public BlockSecretMiddleware(RequestDelegate next)
        {
            _next = next;
        }

        public async Task InvokeAsync(HttpContext context)
        {
            if (context.Request.Path.StartsWithSegments("/secret"))
            {
                context.Response.StatusCode = 403;
```

```
            await context.Response.WriteAsync("❌ Access Denied: Secret path is blocked.");
            return; // stops pipeline
        }

        await _next(context);
    }
}

public static class BlockSecretMiddlewareExtensions
{
    public static IApplicationBuilder UseSecretBlocker(this IApplicationBuilder app)
    {
        return app.UseMiddleware<BlockSecretMiddleware>();
    }
}
```

🧱 Step 3 — Controller

Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;

namespace MiddlewareDemo.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class HomeController : ControllerBase
```

```
    {
        [HttpGet("welcome")]
        public IActionResult Welcome()
        {
            return Ok("Welcome to Middleware Demo API!");
        }

        [HttpGet("data")]
        public IActionResult GetData()
        {
            return Ok(new { Id = 1, Name = "Middleware Example" });
        }
    }
}
```

🧱 Step 4 — Program.cs (The Pipeline)

This is where you configure the middleware order.

Program.cs

```
using MiddlewareDemo.Middlewares;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
```

```
var app = builder.Build();

// ---------------------------------------------
// 🔥 GLOBAL MIDDLEWARE PIPELINE
// ---------------------------------------------

app.UseHttpsRedirection();

// 1️⃣ Custom logging middleware
app.UseRequestLogging();

// 2️⃣ Middleware that blocks /secret route
app.UseSecretBlocker();

// 3️⃣ Built-in middleware for Swagger
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseAuthorization();

// Maps controller endpoints
app.MapControllers();
```

```
// 4️⃣ Terminal Middleware for unmatched routes
app.Run(async context =>
{
    await context.Response.WriteAsync("✓ Pipeline reached the end.");
});

// Run the app
app.Run();
```

🚀 **Step 5 — Run the API and Test Endpoints**

✓ Test 1: Normal route

URL:

https://localhost:5001/home/welcome

Output:

"Welcome to Middleware Demo API!"

Console Logs:

➡ Request: GET /home/welcome

⬅ Response: 200

✓ Test 2: Calling /secret route

URL:

https://localhost:5001/secret

Output:

❌ Access Denied: Secret path is blocked.

✓ Test 3: Non-existing route

URL:

https://localhost:5001/unknown

Output:

✓ Pipeline reached the end.

🎯 What This Project Teaches

✓ How middleware is arranged in a pipeline

✓ Creating custom middleware

✓ Using extension methods for clean registration

✓ Terminal middleware usage

✓ How order affects behavior

✓ Built-in + custom middleware together